

Самостійна робота №10: Кешування та оптимізація продуктивності

Тема: Кешування та оптимізація продуктивності

Мета: Вивчити основи Redis, стратегії кешування, HTTP-кеш заголовки та підходи до оптимізації продуктивності серверних застосунків.

Кількість годин: 3

Теоретичний матеріал

Redis - основи

Redis (Remote Dictionary Server) - in-memory сховище даних типу "ключ-значення". Працює з RAM, що забезпечує надзвичайну швидкість (мікросекунди на операцію). Підтримує різні типи даних: strings, lists, sets, sorted sets, hashes, streams.

Основні команди: `SET key value`, `GET key`, `DEL key`, `EXPIRE key seconds` (TTL), `EXISTS key`. Redis використовується для кешування, сесій, черг, pub/sub, лічильників, rate limiting.

Підключення у Node.js через бібліотеку `ioredis` або `redis`. У Nest.js інтеграція через `@nestjs/cache-manager` з драйвером `cache-manager-redis-store`.

Стратегії кешування

Cache-Aside (Lazy Loading) - застосунок спочатку перевіряє кеш; якщо даних немає (cache miss) - запитує з БД, зберігає у кеш та повертає. Переваги: кешуються тільки потрібні дані. Недоліки: перший запит завжди повільний.

Write-Through - дані записуються одночасно у кеш та БД. Переваги: кеш завжди актуальний. Недоліки: повільніший запис.

Write-Behind (Write-Back) - дані записуються у кеш, а в БД - асинхронно, з затримкою. Переваги: швидкий запис. Недоліки: ризик втрати даних.

TTL (Time-To-Live) - час життя запису у кеші. Після закінчення TTL запис автоматично видаляється. Балансування: короткий TTL - актуальніші дані, довший TTL - менше навантаження на БД.

HTTP-кеш заголовки

Cache-Control - основний заголовок керування кешем. Директиви: `public` / `private`, `max-age=N` (секунди), `no-cache` (перевіряти актуальність), `no-store` (не кешувати взагалі), `immutable` (ніколи не змінюється).

ETag - унікальний ідентифікатор версії ресурсу. Клієнт надсилає `If-None-Match` - сервер порівнює ETag і повертає `304 Not Modified` або нові дані.

Last-Modified / If-Modified-Since - аналогічно ETag, але на основі дати модифікації.

Кешування у Nest.js

Nest.js має вбудований модуль `CacheModule`. Декоратор `@CacheKey()` задає ключ, `@CacheTTL()` - час життя. `CacheInterceptor` автоматично кешує відповіді GET-запитів. Для складніших сценаріїв - пряма робота з `CACHE_MANAGER`.

Контрольні питання

1. Що таке Redis і чому він забезпечує високу швидкість роботи?
2. Які типи даних підтримує Redis?
3. Опишіть стратегію Cache-Aside та її переваги й недоліки.
4. Чим відрізняється Write-Through від Write-Behind?
5. Як працює TTL і як обрати оптимальне значення?
6. Яка різниця між HTTP-заголовками `no-cache` та `no-store`?
7. Як працює механізм ETag для умовних запитів?
8. Як налаштувати кешування у Nest.js за допомогою `CacheModule`?

Порядок виконання

1. Вивчити теоретичний матеріал, наведений у цьому документі.

2. Ознайомитись з рекомендованими джерелами.
3. Відповісти на контрольні питання.

Рекомендовані джерела

- [Redis - офіційна документація](#) - повна документація Redis
- [Redis - Commands](#) - довідник команд Redis
- [MDN - HTTP Caching](#) - гайд по HTTP-кешуванню
- [Cache-Control - MDN](#) - документація заголовка
- [Nest.js - Caching](#) - кешування у Nest.js
- [Caching Strategies - AWS](#) - огляд стратегій кешування